

Testing

VaporTesting

Vapor includes a module named `VaporTesting` that provides test helpers built on `Swift Testing`. These testing helpers allow you to send test requests to your Vapor application programmatically or running over an HTTP server.

Note

For newer projects or teams adopting Swift concurrency, `Swift Testing` is highly recommended over `XCTest`.

Getting Started

To use the `VaporTesting` module, ensure it has been added to your package's test target.

```
let package = Package(  
    ...  
    dependencies: [  
        .package(url: "https://github.com/vapor/vapor.git", from: "4.110.1")  
    ],  
    targets: [  
        ...  
        .testTarget(name: "AppTests", dependencies: [  
            .target(name: "App"),  
            .product(name: "VaporTesting", package: "vapor"),  
        ])  
    ]  
)
```

Warning

Be sure to use the corresponding testing module, as failing to do so can result in Vapor test failures not being properly reported.

Then, add `import VaporTesting` and `import Testing` at the top of your test files. Create structs with a `@Suite` name to write test cases.

```
@testable import App  
import VaporTesting  
import Testing  
  
@Suite("App Tests")  
struct AppTests {  
    @Test("Test Stub")  
    func stub() async throws {  
        // Test here.  
    }  
}
```

Each function marked with `@Test` will run automatically when your app is tested.

To ensure your tests run in a serialized manner (e.g., when testing with a database), include the `.serialized` option in the test suite declaration:

```
@Suite("App Tests with DB", .serialized)
```

Testable Application

Define a private method function `withApp` to streamline and standardize the setup and teardown for our tests. This method encapsulates the lifecycle management of the `Application` instance, ensuring that the application is properly initialized, configured, and shut down for each test.

In particular it is important to release the threads the application requests at startup. If you do not call `asyncShutdown()` on the app after each unit test, you may find your test suite crash with a precondition failure when allocating threads for a new instance of `Application`.

```
private func withApp(_ test: (Application) async throws -> ()) async throws {
    let app = try await Application.make(.testing)
    do {
        try await configure(app)
        try await test(app)
    }
    catch {
        try await app.asyncShutdown()
        throw error
    }
    try await app.asyncShutdown()
}
```

Pass the `Application` to your package's `configure(_:)` method to apply your configuration. Then you test the application calling the `test()` method. Any test-only configurations can also be applied.

Send Request

To send a test request to your application, use the `withApp` private method and inside use the `app.testing().test()` method:

```
@Test("Test Hello World Route")
func helloWorld() async throws {
    try await withApp { app in
        try await app.testing().test(.GET, "hello") { res async in
            #expect(res.status == .ok)
            #expect(res.body.string == "Hello, world!")
        }
    }
}
```

The first two parameters are the HTTP method and URL to request. The trailing closure accepts the HTTP response which you can verify using `#expect` macro.

For more complex requests, you can supply a `beforeRequest` closure to modify headers or encode content. Vapor's [Content API](#) is available on both the test request and response.

```
let newDTO = TodoDTO(id: nil, title: "test")

try await app.testing().test(.POST, "todos", beforeRequest: { req in
    try req.content.encode(newDTO)
}, afterResponse: { res async throws in
    #expect(res.status == .ok)
    let models = try await Todo.query(on: app.db).all()
    #expect(models.map({ $0.toDTO().title }) == [newDTO.title])
})
```

Testing Method

Vapor's testing API supports sending test requests programmatically and via a live HTTP server. You can specify which method you would like to use through the `testing` method.

```
// Use programmatic testing.
app.testing(method: .inMemory).test(...)

// Run tests through a live HTTP server.
app.testing(method: .running).test(...)
```

The `inMemory` option is used by default.

The `running` option supports passing a specific port to use. By default `8080` is used.

```
app.testing(method: .running(port: 8123)).test(...)
```

Database Integration Tests

Configure the database specifically for testing to ensure that your live database is never used during tests.

```
app.databases.use(.sqlite(.memory), as: .sqlite)
```

Then you can enhance your tests by using `autoMigrate()` and `autoRevert()` to manage the database schema and data lifecycle during testing:

By combining these methods, you can ensure that each test starts with a fresh and consistent database state, making your tests more reliable and reducing the likelihood of false positives or negatives caused by lingering data.

Here's how the `withApp` function looks with the updated configuration:

```
private func withApp(_ test: (Application) async throws -> ()) async throws {
    let app = try await Application.make(.testing)
    app.databases.use(.sqlite(.memory), as: .sqlite)
    do {
        try await configure(app)
        try await app.autoMigrate()
        try await test(app)
        try await app.autoRevert()
    }
    catch {
        try? await app.autoRevert()
        try await app.asyncShutdown()
        throw error
    }
    try await app.asyncShutdown()
}
```

XCTVapor

Vapor includes a module named `XCTVapor` that provides test helpers built on `XCTest`. These testing helpers allow you to send test requests to your Vapor application programmatically or running over an HTTP server.

Getting Started

To use the `XCTVapor` module, ensure it has been added to your package's test target.

```
let package = Package(
    ...
    dependencies: [
        .package(url: "https://github.com/vapor/vapor.git", from: "4.0.0")
    ],
    targets: [
```

```

    ...
    .testTarget(name: "AppTests", dependencies: [
        .target(name: "App"),
        .product(name: "XCTVapor", package: "vapor"),
    ])
  ]
)

```

Then, add `import XCTVapor` at the top of your test files. Create classes extending `XCTestCase` to write test cases.

```

import XCTVapor

final class MyTests: XCTestCase {
    func testStub() throws {
        // Test here.
    }
}

```

Each function beginning with `test` will run automatically when your app is tested.

Testable Application

Initialize an instance of `Application` using the `.testing` environment. You must call `app.shutdown()` before this application deinitializes.

The shutdown is necessary to help release the resources that the app has claimed. In particular it is important to release the threads the application requests at startup. If you do not call `shutdown()` on the app after each unit test, you may find your test suite crash with a precondition failure when allocating threads for a new instance of `Application`.

```

let app = Application(.testing)
defer { app.shutdown() }
try configure(app)

```

Pass the `Application` to your package's `configure(_:)` method to apply your configuration. Any test-only configurations can be applied after.

Send Request

To send a test request to your application, use the `test` method.

```

try app.test(.GET, "hello") { res in
    XCTAssertEqual(res.status, .ok)
    XCTAssertEqual(res.body.string, "Hello, world!")
}

```

The first two parameters are the HTTP method and URL to request. The trailing closure accepts the HTTP response which you can verify using `XCTAssert` methods.

For more complex requests, you can supply a `beforeRequest` closure to modify headers or encode content. Vapor's [Content API](#) is available on both the test request and response.

```

try app.test(.POST, "todos", beforeRequest: { req in
    try req.content.encode(["title": "Test"])
}, afterResponse: { res in
    XCTAssertEqual(res.status, .created)
    let todo = try res.content.decode(Todo.self)
    XCTAssertEqual(todo.title, "Test")
}))

```

Testable Method

Vapor's testing API supports sending test requests programmatically and via a live HTTP server. You can specify which method you would like to use by using the `testable` method.

```
// Use programmatic testing.  
app.testable(method: .inMemory).test(...)  
  
// Run tests through a live HTTP server.  
app.testable(method: .running).test(...)
```

The `inMemory` option is used by default.

The `running` option supports passing a specific port to use. By default `8080` is used.

```
.running(port: 8123)
```